Deanna Turner
4/15/26
Prof. Sloan
CS 5100

Using Logistic Regression to Predict the Winner of the 2026 Stanley Cup

I - Introduction

Within the realm of hockey specifically, there is an abundance of data that can be used to create various models, but few true Stanley cup prediction models exist. Understandably so, given that it's a hard thing to predict. Hockey is understood to be way more of a game of luck than many other professional sports. No amount of advanced stats can really capture the luck factor and "vibes", for lack of a better term, that may help a team win the Stanley cup. Additionally, the Stanley Cup is argued to be the hardest trophy to win in professional sports, given the rigorous four-round playoffs that could see teams play anywhere from 16 to 28 games. The unexpected can absolutely happen too. Even if a team is the highest seeded, there's an acknowledged "President's Trophy" curse (given to the team who scores the most points during the regular season). Teams that people have totally written off as having no business being in the playoffs have also taken down juggernaut teams and gone on so-called "Cinderella" cup runs.

The current Stanley Cup playoff format was instantiated in 2020. The NHL has two conferences (east and west) and each conference has two divisions. In this format, there are 16 playoff teams, where the top three in each division advance, as well as two wild card spots per conference. Each division winner (D1) plays the lower-seeded wild card (WC2), and the division runner up

(D2) plays the division third place finisher (D3).

The paper that is the basis for this work [1] determined that among various machine learning techniques, logistic regression worked the best for predicting the outcome of a single game. In this project, logistic regression is used to help predict, based on the current playoff matchups, the winners of the first round series, and then so on and so forth to predict a cup winner. The data used to train the model consists of historical playoff data, regular season stats and head-to-head matchups including advanced metrics to more fully evaluate a team's on-ice play, and regular season goalie statistics. The logistic regression model from sklearn was used for this task, and training was executed. Performance was evaluated via Monte Carlo simulations that took into account generated features per matchup and the resulting probability that a certain team would win as a result. 10,000 Monte Carlo simulations were run on the full bracket (after combining the Eastern conference and Western conference matchup sets).

Based on the training of the model, the prediction model says that the winner of the 2026 Stanley Cup will be the Colorado Avalanche, as they frequently won in a majority of the Monte Carlo simulations. The model was also backtested on previous NHL seasons to predict the cup winners, but struggled to do this. Overall, the model struggled with accuracy due to issues with data collection, flattening, and overall feature extraction.

II - Methods

Data Collection

Data was gathered from three different sources: Kaggle datasets, NHL's API, and moneypuck.com [3]. The Kaggle datasets aggregate playoff data between 1985 and

2025 [2], with one sheet containing game level information (participating teams, goals for each team, shots for each team) and the other contains skater boxscore information (team, player name, jersey number, time one ice, goals and assists, shots, powerplay goals, penalty minutes, etc.). These sheets were cleaned to only contain data from 2020 onwards, to coincide with the new playoff format. This is the core source of playoff data from which features for training the model are built.

The next big chunk of data comes from NHL's API, funneled through the nhl-api-py package, which makes interfacing with the endpoints much easier. It's used to grab the current standings data (for creating the final aggregated dataset), current playoff bracket (which, by the time of having written this paper, is set and not in flux like during the development of this project), and the team stats data for a given season (it's called for each of the six seasons of data used for training). This data is all regular season data, spanning from 2020-21 to 2025-26 (the current season). The team stats data consists of a team's faceoff win %, games played (typically 82 except for the case of the COVID-shortened 2020-21 season, which was 56 games), goals against, goals against per game, goals for, goals for per game, overtime losses, penalty kill net %, penalty kill %, point %, points, powerplay net %, powerplay %, regulation and overtime wins (ROW), shots against per game, shots for per game, shutouts, ties, wins, regulation wins, and shootout wins in the regular season. This is eventually used to help construct features about a team's goal differential (how many goals they score versus are scored on them), win percentage, shots per game (which measures offensive production value), shots against per game (which measures defensive production value, typically less shots against signals the

defense is doing their job well), powerplay percentage (how often does a team score on the man advantage), and penalty kill percentage (how often does a team kill off penalties they've taken).

The third chunk of data comes from moneypuck.com's [3] datasets. This was necessary, as the NHL API is rather primitive in the advanced stats that can be collected from it. On the team level per season, Moneypuck has data pertaining to Corsi percentage, expected goals for per 60 min (xGF), expected goals against per 60 min (xGA), expected goals for % (xGF%), high danger chances for per 60 min (HDCF), high danger chances against for 60 min (HDCA), goalie save percentage (SV%), and goals saved above expected (GSAx). Corsi percentage uses shots on goal, missed shots on goal, and blocked shot attempts to measure shot attempt differential at even strength, and is functionally a measure of a team's puck possession. xGA and xGF are based on shot attempts weighted for shot quality using shot type, location, angle, or if a shot was off a rebound/off the rush. It's essentially the probability of scoring on a shot based on the listed factors [4]. HDCF and HDCA are shot attempts in the mid to low slot range. SV% is a measure of how many saves did a goalie make/how many were let in. GSAx is a measure of goalie performance using the actual goals against in a game compared to the xGA (xGA - GA). These advanced stats give more information about a team's overall play and will sway the outcome of games. More data than these columns are included in Moneypuck's datasets, but these are the ones being used for feature extraction specifically.

Processing

As for the processing, the file team_stats.py contains code to pull the aforementioned

team stats data between the 2020-21 season and the 2025-26 season. There's two interesting edge cases when pulling this data, though. The Seattle Kraken's inaugural season was in 2021-22, so no data exists for them in 2020-21. The Utah Mammoth's inaugural season was in 2024-25, and thus no data exists for them before that point. There is data for the Arizona Coyotes, a team whose last season was in 2023-24 before being relocated to Utah, but they are considered two separate franchises. For this reason, the Arizona data is not aggregated with the Utah data. This was a specific design choice in the context of this project, and potentially skews some of the training data. The following raw columns are extracted from this call to the season stats endpoint: team, games played, wins, losses, overtime losses, points, goals for, goals against, powerplay %, penalty kill %, shots per game and shots against per game. The data from each API call for the necessary seasons is cached in a CSV file in the format "nhl_team_summary_{season}", so that rate limiting doesn't occur while running the pipeline. Once the raw data is collected, it is used to build a dataset containing the raw columns and some derived ones, those being win % (calculated using wins divided by games played), and goal differential per game (which is total goals for minus total goals against divided by games played). This is used later for constructing a final dataset. advanced_stats.py contains code to pull the aforementioned data from [3]. There is a function to pull goalie stat data and team stat data, each of which caches the data in a CSV file to avoid rate limiting and redundancy (files for goalie stats are named "moneypuck_goalie_{season}" and "moneypuck_team_{season}"). The goalie CSVs have the team, season, GSAx, and SV% aggregated for all goalies who played for the team in the given season. The team CSVs have the advanced stats mentioned

previously (Corsi %, xGF, xGA, xGF%, HDCF, HDCA, and more that go unused). A function to produce a dataframe combining the data from these two categories also exists for later purposes of constructing a final, joined dataset.

build_full_dataset.py joins the NHL derived dataframe with the MoneyPuck derived dataframe, weighting certain features based on how recent the season that data came from was. The weighting scheme is as follows:

```
SEASON_WEIGHTS = {
        "20252026": 1.0,
        "20242025": 0.8,
        "20232024": 0.6,
        "20222023": 0.4,
        "20212022": 0.2,
        "20202021": 0.1
}
```

This was done to incentivize better performance in recent seasons, as that is generally (but not always) a better indicator of success. The two dataframes are merged, with some extra columns added for team identification. Additional advanced features (xGF/60, xGA/60, HDCF/60, HDCA/60, xGF%) which have to be calculated based on the columns from MoneyPuck, are also joined to this "full" dataset, and weighting is applied to the necessary data points. The output of this is saved to a CSV called full_dataset.csv.

Once that is constructed, the dataframe containing the full regular season dataset, the dataframe containing playoff game data, the list of seasons, the season weights, and the names of the two teams in a matchup are fed to a function that builds the features for each given matchup in the computed playoff bracket, so that the model can train on specific matchup data. The build_features function has a four-step approach. First, features for team strength for each team weighted across the seasons is calculated. This includes wins % differential, goal differential per game differential, shots for

differential, shots against differential, powerplay % differential and penalty kill % differential. Next, features for team strength based on advanced stats are calculated for each team. These features include xGF differential, xGA differential, xGF% differential, Corsi differential, HDCF differential, SV% differential and GSAx differential. Next, features derived from regular season head-to-head matchups, summarized across the seasons, are added. This includes the amount of head-to-head games for each team (which is higher for teams in the same division, and lower for teams across conferences), team 1's head-to-head wins, the goal differential, home win rate and away win rate (all from team 1's perspective). Lastly, features based on previous playoff matchups between the two teams are considered. This includes information about how many series the teams have played against each other, and how many wins team 1 has. This list of features is constructed, and then passed to the model for training. It is also saved in a CSV file called model_input_{date}.csv. As an aside, this had to be removed from the final set of features due to extreme difficulty wrangling with conflicting data formats, but the intent was to have this in there as a stronger set of features.

Model Architecture and Training

The model used in this project was a logistic regression model from sklearn. A training dataset is constructed in build_training_dataset.py, and this is used to train the model before running any predictions. This dataset includes the same full dataset, playoff dataset, seasons and season weights inputs. logistic_regression_model.py also contains several functions that help aid the Monte Carlo simulations, as well as the Monte Carlo function itself. There is a function that creates the probability of a team winning

based on computed matchup features, and this is cached so that the function does not keep generating duplicate data. The simulate_playoffs_dynamic function basically dynamically recomputes features and win probabilities for each round matchup. It is fed the Eastern and Western conference matchups separately due to issues with Eastern conference data winning out for some odd reasons. There are two Monte Carlo functions; the progression one is basically used for evaluation later on, and graphs the probabilities over specified intervals. The Monte Carlo bracket function is used to unify the predictions from the Eastern and Western conference matchup data.

Running the Whole Pipeline

The file run_pipeline.py runs all data processing functions from start to finish. First, it instantiates the nhl-api-py client and loads the playoff CSV data into a dataframe. Then, the current standings are generated, the playoff bracket is generated, and the first round matchups (that are guaranteed at this point) are extracted for further use. Next, the full joined dataset is built and saved to a CSV file. Next, features are built for each first-round matchup. Then the training dataset is built and training is executed on the model. Due to the nature of the data structure, it was not possible to run Monte Carlo simulations on the whole bracket as one object, so the Eastern conference and Western conference matchups are assembled separately, and then passed into the Monte Carlo bracket simulation together. It was necessary to split up the data this way, because for a majority of testing trials, the model was only outputting Eastern conference data, and the Western conference data was getting lost. The eastern results/champ and western results/champ are assembled from the dynamic playoff simulation, and then final features are

computed and fed to the model, which then predicts the eventual cup winner. Oftentimes, the printout of the east/west winners and the cup champion don't align with the resulting graphs. Then, 10,000 Monte Carlo simulations are run on the joint east/west matchup data, and the results are graphed on which team was a) most likely to win their respective conference and b) most likely to win the Stanley Cup overall.

III - Results

Predicting the 2026 Eastern and Western Conference Winners, as well as Stanley Cup Winners

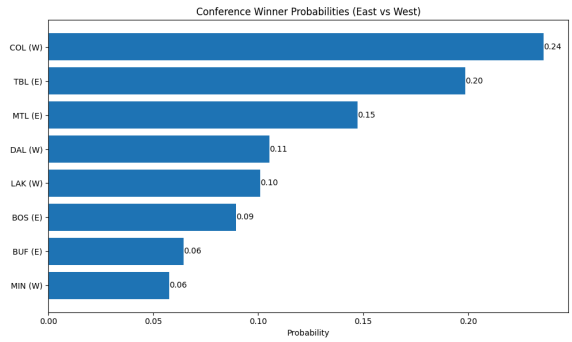

Figure 1. Conference Winner Probabilities (East vs. West)

After running the 10,000 full bracket simulations, the Colorado Avalanche was given the highest chance (24%) to win the Western conference title. The Tampa Bay Lightning were given a 20% chance to win the Eastern conference title. Other strong suitors for their respective conference titles included the Montréal Canadiens (east), Dallas Stars (west), Los Angeles Kings (west), Boston Bruins (east), Buffalo Sabres (east), and Minnesota Wild (west).

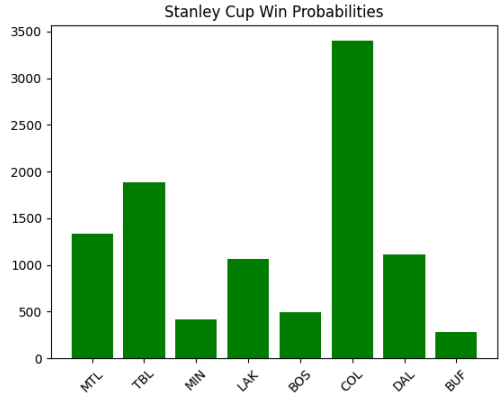

Figure 2. Stanley Cup Win Probabilities

Figure 2 shows out of the 10,000 simulations, the frequency in which the top eight teams won the simulation. The Colorado Avalanche topped out at the most won simulations, followed by the Tampa Bay Lightning, Montréal Canadiens, Dallas

Stars, Los Angeles Kings, Boston Bruins, Minnesota Wild and Buffalo Sabres.

Accuracy for the model itself was hovering between 0.67 and 0.53. This is a very very steep drop from when it was originally being trained, and this is due to the fact that head to head matchup features (for regular and playoff games) had to be removed, severely affecting the accuracy of the model.

Predicting the last five Stanley Cup Winners

As a benchmark to test the accuracy of the features gathered, the model was told to predict the winner of the Stanley Cup in 2021, 2022, 2023, 2024, and 2025. These winners are already known. evaluate_historical.py is a standalone script used to run this evaluation, printing out the model accuracy for each year, the actual cup winner, the predicted cup winner, and whether or not the model was correct.

```
Evaluating 2021...
Accuracy: 0.7333333333333333

Evaluating 2022...
Accuracy: 0.5333333333333333

Evaluating 2023...
Accuracy: 0.4

Evaluating 2024...
Accuracy: 0.5333333333333333

Evaluating 2025...
Accuracy: 0.6666666666666666
   year predicted actual  correct
0  2021       TBL    TBL     True
1  2022       CAR    COL    False
2  2023       CAR    VGK    False
3  2024       CAR    FLA    False
4  2025       CAR    FLA    False
Accuracy 0.2
```

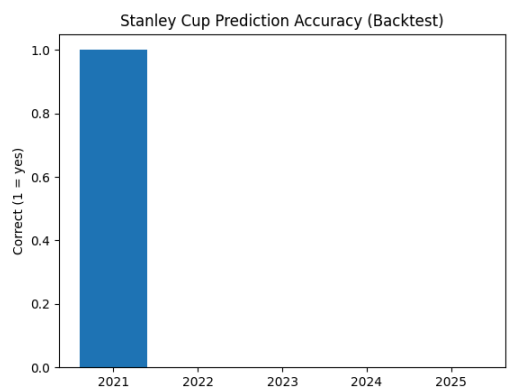

Figure 3. Stanley Cup Prediction Accuracy Backtest

Figure 3 shows the model was only accurate in predicting the 2021 cup winner, and the printouts suggest the model had decent

accuracy but failed to predict the right winner. This shows that the model not only struggles to predict this year's cup winner, but also struggles to predict previous cup winners most of the time.

IV - Conclusions

Overall, the features were flattened and were used to train a model that generally outputted the same predictions (or similar) each time. The model didn't perform very well, objectively, and thus much tuning could be done to all parts of it. There were a lot of things that went wrong, however, that are explained in detail in this section.

As for things that could've gone better with this approach, the main thing is the gathering and joining of the data itself. The data used to train the prediction model is objectively limited, given it only spans six seasons worth of play. Future iterations of this could use data from more seasons to improve performance. Additionally, more playoff data could've been accumulated, such as the same advanced stats and goalie-specific stats that were collected for the regular season, as well as other factors such as home ice advantage, venue adjusted scoring, last 10 games records, and previous playoff success (as in, if a series went to a game 6 or 7, which team has historically won more of those). Home ice advantage plays a decent part in the outcomes of certain games and is a factor that would likely improve model performance. A team's record in their last 10 games is important enough, as it shows what teams are on a hot streak or a skid, and getting "hot" at the right time is undoubtedly important for a postseason that requires a lot of longevity. As for addressing the joining of data issue, it's quite difficult to flatten a large amount of data for various skaters and goalies on a team into as few meaningful rows as possible. The data is highly dynamic;

players come and go from teams due to trades, injuries, call ups to the NHL/demotions to the minor leagues, etc., and this greatly affects what a team looks like on paper. Goalie performance is highly volatile, in the sense that one month they may save many more goals than they are expected to, and the next month they let in a ton of goals.

Also, it's worth noting there's no way to represent things off the ice that can affect a team's performance, and there's no good way to strictly model "puck luck". Team morale is huge, and when players aren't in the right headspace, they may not perform very well. Coaching changes can affect that, as well as losing certain players to trade/injury. Bad puck luck sends players into frustrating cold streaks where oftentimes, their entire game breaks down. Good puck luck might mean that a player is shooting at a very high percentage and making many of his shots, while his career average might be much lower than that, indicating luck-based goals. This isn't necessarily a flaw of this approach, but it's something that has to be kept in mind with any sort of hockey analytics model.

One more improvement that could've been made is related to the weighting of certain data points to derive features from. The only weights used were on the seasons, with the most recent season (2025-26) being weighted the highest, and the least recent season (2020-21) the lowest. While this was good in terms of prioritizing certain data over other data, this could've been extrapolated to weight certain advanced stats and normal stats higher than others (effectively deeming them as more impactful to the game result). Going forward, more of these statistics should be weighted differently, as it would likely improve the output of the model.

One shortcoming of this approach that happened right at the completion of the project was the total mismatch in data needed to project head to head matchups, both in the regular season and in the playoffs. These features ended up having to be removed, as there was no way the pipeline would be able to complete despite many attempts to work through the various issues. The crux of the issue boils down to the sheer inconsistency of data formats for the NHL between vendors that make it very difficult to join what is needed.

Works Cited

[1] J. S. Chin, F. H. Juwono, I. M. Chew, S. Sivakumar and W. K. Wong, "Predicting Ice Hockey Results Using Machine Learning Techniques," 2023 International Conference on Digital Applications, Transformation & Economy (ICDATE), Miri, Sarawak, Malaysia, 2023, pp. 1-5, doi: 10.1109/ICDATE58146.2023.10248726.

[2] "MoneyPuck.com -Download Datasets," *moneypuck.com*. https://moneypuck.com/data.htm

[3] ryanhdar, "NHL Team Data Per Season + Playoff Appearances," *Kaggle.com*, 2026. https://www.kaggle.com/datasets/ryanhdar/nhl-team-data-and-playoff-result (accessed Apr. 15, 2026).

[4] "A brief guide to hockey's advanced statistics," *Second City Hockey*, May 25, 2022. https://www.secondcityhockey.com/advanced-hockey-statistics-explainer-faqs-walkthroughs/ (accessed Apr. 15, 2026).

Libraries used: nhl-api-py (nhlpy), numpy, matplotlib, pandas, os, datetime, sklearn, functools, collections, requests, io, time